

**Universidad Autonoma de Santo Domingo**

Facultad de Ingenieria y Arquitectura

Asignatura: INF-8237 Ciencia de Datos I

## **Caso practico 2**

**CRUD/ORM nativo basado en POO y estructuras de datos en MySQL Sakila**

Equipo: UASDVirtual

Integrante: Marlenis Judith Concepcion Cuevas

Facilitador: [Completar]

Fecha: 4 de junio de 2026

## **Resumen**

Este informe presenta el desarrollo del Caso practico 2 de la Unidad 2 de Ciencia de Datos I, consistente en la creacion de un CRUD/ORM nativo en Python para operar sobre la base de datos Sakila de MySQL. La solucion utiliza programacion orientada a objetos para representar entidades relacionales como paises, ciudades, peliculas e inventario mediante clases de dominio, repositorios, controladores, servicios y un contexto de base de datos. El proyecto incorpora estructuras de datos como listas de entidades, diccionarios para cache y un historial de operaciones para evidenciar el uso de abstracciones propias de Python. Para facilitar la ejecucion por los integrantes del equipo, se prepararon scripts de inicio para Mac, Linux y Windows que levantan MySQL en Docker, importan Sakila, verifican la conexion y abren el menu interactivo del CRUD. Ademas del flujo crear, leer, actualizar y eliminar, el trabajo incluye importacion y exportacion en CSV y JSON, diez consultas SQL, restricciones de integridad mediante unique constraints y metricas descriptivas fundamentales como media, rango, desviacion estandar, varianza y covarianza. Los resultados demuestran una arquitectura modular y escalable, cercana al patron ORM, donde los cambios del modelo pueden mantenerse separados de la conexion, la logica de aplicacion y la interfaz de consola. La propuesta fortalece la comprension practica de bases de datos relacionales, consumo programatico de datos y organizacion de codigo reutilizable para proyectos de ciencia de datos.

## **Introduccion**

La manipulacion de datos en ciencia de datos requiere comprender tanto la estructura de los datos como los mecanismos usados para consultarlos, transformarlos y validarlos. En este caso practico se utiliza Sakila, una base de datos de ejemplo de MySQL, para construir una solucion que conecta Python con un sistema gestor relacional y permite ejecutar operaciones CRUD desde una interfaz de consola.

La actividad responde al objetivo de crear un CRUD/ORM nativo basado en programación orientada a objetos y estructuras de datos. El enfoque ORM permite representar tablas como objetos de dominio y encapsular las operaciones SQL en repositorios y servicios, reduciendo la dependencia directa entre la interfaz de usuario y las consultas de base de datos (Fowler, 2002).

## **Marco de referencia**

Un CRUD agrupa las operaciones create, read, update y delete, necesarias para administrar registros persistentes. En bases de datos relacionales, estas acciones se implementan mediante instrucciones SQL como INSERT, SELECT, UPDATE y DELETE. MySQL documenta Sakila como una base de datos de ejemplo útil para practicar relaciones entre tablas de un sistema de alquiler de películas (Oracle, s.f.).

Python facilita este tipo de solución por su soporte para clases, dataclasses, listas, diccionarios y módulos reutilizables. La documentación oficial de Python describe las clases como un mecanismo para combinar datos y comportamiento dentro de una misma estructura (Python Software Foundation, s.f.). Docker, por su parte, permite ejecutar servicios como MySQL en contenedores reproducibles, lo que reduce diferencias entre equipos de trabajo (Docker, s.f.).

## **Descripción del caso práctico**

El proyecto gestiona las entidades country, city, film e inventory de Sakila. Estas entidades cubren los requisitos de países, ciudades, películas e inventario planteados en la asignación. La conexión se configura mediante variables de entorno y puede ejecutarse con MySQL local o con el contenedor Docker preparado para el proyecto.

## **Arquitectura del sistema**

- db.py: administra la conexión a MySQL y el manejo transaccional.
- dbcontext.py: centraliza repositorios, cache e historial.

- models.py: define entidades y ModelCollection como lista de objetos.
- repositories.py: implementa CRUD generico y repositorios concretos.
- controllers.py y services.py: separan el flujo de aplicacion de la persistencia.
- structures.py: contiene cache de entidades e historial de consultas.
- metrics.py y reports.py: calculan media, rango, varianza, desviacion y covarianza.
- import\_export.py: permite importar y exportar datos CSV y JSON.
- main.py: ofrece el menu de consola para ejecutar la demostracion.

## **Implementacion del CRUD**

La capa BaseRepository recibe una clase de modelo y construye consultas parametrizadas para crear, buscar por identificador, listar, actualizar y eliminar registros. Cada repositorio concreto hereda esa funcionalidad para una entidad especifica. La interfaz de consola permite seleccionar paises, ciudades, peliculas e inventario, introducir datos y observar el resultado devuelto como objeto.

## **Pruebas y resultados**

Las pruebas unitarias disponibles validan estructuras de datos usadas por el proyecto. En la verificacion local se ejecuto pytest sobre el Caso 2 y el resultado fue de dos pruebas aprobadas. Tambien se comprobe la conexion a MySQL Sakila por Docker, obteniendo conexion exitosa, nombre de base de datos sakila y version MySQL 8.0.46.

- Create: creacion de paises de prueba desde el menu.
- Read: busqueda por ID y listado de entidades.
- Update: modificacion de registros seleccionados.
- Delete: eliminacion de registros de prueba.
- Metricas: calculo descriptivo sobre longitud, tarifa y costo de peliculas.
- SQL: ejecucion de diez consultas y script de unique constraints.

## Conclusiones

La solución cumple el objetivo principal de implementar un CRUD/ORM nativo sobre Sakila usando POO y estructuras de datos. La separación entre contexto, modelos, repositorios, controladores y servicios facilita ampliar el sistema a nuevas tablas sin reescribir toda la aplicación. La automatización con Docker mejora la reproducibilidad para los integrantes del equipo y permite generar evidencias reales de ejecución. Como mejora futura, se recomienda ampliar las pruebas de integración y agregar validaciones de negocio más detalladas antes de insertar o actualizar datos.

## Referencias

Docker. (s.f.). Docker documentation. <https://docs.docker.com/>

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Oracle. (s.f.). Sakila sample database. MySQL documentation.  
<https://dev.mysql.com/doc/sakila/en/>

Python Software Foundation. (s.f.). The Python tutorial.  
<https://docs.python.org/3/tutorial/>

## Anexos

- Anexo A. Código fuente relevante: carpeta src/ del Caso práctico 2.
- Anexo B. Evidencias de ejecución: capturas de conexión, CRUD, métricas y pruebas.
- Anexo C. Scripts SQL: consultas y unique constraints incluidos en la carpeta sql/.
- Anexo D. Uso de agentes IA: apoyo en documentación, automatización y revisión.